

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
KALANKI, KATHMANDU



[CT755]
A MAJOR PROJECT REPORT ON
“SMART LOGISTIC TRASPORTATION USING
REINFORCEMENT LEARNING”

Submitted By:

Aayush Basnet	ACE077BCT004
Ankrit Risal	ACE077BCT013
Bhuwan Dhakal	ACE077BCT024

Project Supervisor:

Associate Professor Dr. Dibakar Raj Pant

A Major Project Final report submitted to the Department of Electronics and
Computer Engineering in the partial fulfillment of the requirements for degree of
Bachelor of Engineering in Computer Engineering

Kathmandu, Nepal

2025/02/14

**“SMART LOGISTIC TRASPORTATION USING
REINFORCEMENT LEARNING”**

Submitted by:

Aayush Basnet	ACE077BCT004
Ankrit Risal	ACE077BCT013
Bhuvan Dhakal	ACE077BCT024

Supervised by:

Dr. Dibakar Raj Pant
Associate Professor

**A MAJOR PROJECT SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENT FOR THE DEGREE OF BACHELOR IN COMPUTER OR
ELECTRONICS & COMMUNICATION ENGINEERING**

Submitted to:

"DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING"
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
Balkhu, Kathmandu

2025/02/14

LETTER OF APPROVAL

The undersigned certify that they have read and recommended to the Institute of Engineering for acceptance, a project report entitled “**SMART LOGISTIC TRASPORTATION USING REINFORCEMENT LEARNING**” submitted by:

Aayush Basnet ACE077BCT004

Ankrit Risal ACE077BCT013

Bhuwan Dhakal ACE077BCT024

In the partial fulfillment of the requirements for the degree of Bachelor’s Degree in Computer Engineering.

.....

Project Supervisor

Associate Professor Dr. Dibakar Raj Pant
Department of Electronics and Computer Engineering
IOE Campus, Pulchowk

.....

Head of Department

Senior Asst. Prof. Prem Chandra Roy
Department of Electronics and Computer Engineering

2025/02/14

COPYRIGHT

The author has agreed that the library, Advanced College of Engineering and Management, may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the project work recorded herein or, in their absence, by the Head of the Department wherein the project was done. It is understood that recognition will be given to the report's author and the Department of Electronics and Computer Engineering, Advanced College of Engineering and Management for any use of the material of this project report. Copying publication or the other use of this project for financial gain without the approval of the Department an author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in should be addressed to:

Head of Department
Department of Electronics and Computer Engineering
Advanced College of Engineering and Management
Balkhu, Kathmandu
Nepal

ACKNOWLEDGMENT

We would like to express our profound gratitude and deep regards to our respected supervisor, Associate Professor **Dr. Dibakar Raj Pant**, for his insightful advice, motivating suggestions, invaluable guidance, help, and support in the successful completion of this project and also for his constant encouragement and advice throughout our Bachelor's program.

We express our deep gratitude to **Er. Prem Chandra Roy**, Head of the Department of Electronics and Computer Engineering, **Er. Dhiraj Pyakurel**, Deputy Head, Department of Electronics and Computer Engineering, **Er. Laxmi Prasad Bhatt**, Academic Project Coordinator, Department of Electronics and Computer Engineering, for their support, co-operation, and coordination.

The in-time facilities provided by the department throughout the Bachelor's program are also equally acknowledgeable.

We would like to convey our thanks to the teaching and non-teaching staff of the Department of Electronics & Communication and Computer Engineering, ACEM for their invaluable help and support throughout Bachelor's Degree. We are also grateful to all our classmates for their help, encouragement, and invaluable suggestions.

Finally, yet more importantly, we would like to express our deep appreciation to our grandparents, parents, and siblings for their perpetual support and encouragement throughout the Bachelor's degree period.

Project Members:

Aayush Basnet	ACE077BCT004
Ankrit Risal	ACE077BCT013
Bhuwan Dhakal	ACE077BCT024

ABSTRACT

Logistic transportation is an important part of our lives. From delivering household goods for daily use purposes to delivering important commodities like healthcare supplies, construction materials, document deliveries, etc. logistic transportation plays a significant role in ensuring safe and timely delivery of supplies. With the advancement of machine learning algorithms, enabling trained intelligent agents to navigate and deliver logistics efficiently and effectively thus opening new possibilities in the field of logistic transportation. The project is focused on the use of reinforcement learning, a paradigm of machine learning, where the agents learn to navigate and conduct delivery through trial and error similar to the way we humans learn. The project proposed a multi agent grid-based environment of size 10 x 10 consisting of 3 agents, 3 goals, 3 objects and obstacles with the objective of making the agents pickup their respective objects and delivering them to their respective goals independently. The Q-learning algorithm is implemented to train the agents in developing the optimal policies for navigating in the environment avoiding the obstacles. Learning is achieved through trial and error which gradually converged the strategy of the agent to minimize the delivery time and avoid collisions. The project incorporated independent Q-learning over 10,000 training episodes successfully developing an optimal strategy for efficient pickup and delivery. The visual results indicated the trained agents completing their tasks effectively with optimized routes and reduced delays. Thus, illustrating the use of reinforcement learning based logistics solutions in improving the efficiency of the transportation system along with opening better ways in scaling, optimizing and adapting the everchanging logistic dynamics.

Keywords: *Intelligent agents, Logistic transportation, Machine learning, Optimal policy, Q-learning, Reinforcement learning,*

TABLE OF CONTENTS

Title	Page
LETTER OF APPROVAL	II
COPYRIGHT.....	III
ACKNOWLEDGMENT.....	IV
ABSTRACT.....	V
TABLE OF CONTENTS.....	VI
LIST OF FIGURES	VIII
LIST OF EQUATIONS	IX
LIST OF ABBREVIATIONS.....	X
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.1.1 Intelligent agent	1
1.1.2 Reinforcement learning.....	1
1.1.3 The Agent Environment Interface and the Markov Decision Process (MDP)	2
1.1.4 Reward	2
1.2 Motivation.....	2
1.3 Problem Statement	2
1.4 Objective	3
1.5 Significance of the study.....	4
CHAPTER 2: LITERATURE REVIEW	5
CHAPTER 3: REQUIREMENT ANALYSIS.....	7
3.1 Functional Requirements	7
3.2 Non-functional Requirements	7
CHAPTER 4: METHODOLOGY	8
4.1 Environment setting	8
4.1.1 Single agent system.....	9
4.1.2 Single agent framework	9

4.1.3 State space.....	9
4.1.4 Action space.....	9
4.1.5 Agent policy.....	9
4.1.1 Reward	9
4.2 Q-learning algorithm.....	10
4.3 Multi agent reinforcement learning	11
4.3.1 Independent Q learning.....	11
CHAPTER 5: RESULTS AND ANALYSIS	12
5.1 Result	12
5.1.1 Before Training.....	13
5.1.2 After Training	14
5.2 Analysis.....	16
CHAPTER 6: CONCLUSION, LIMITATIONS AND FUTURE ENHANCEMENT	17
6.1 Conclusion	17
6.2 Limitations	17
6.3 Future Enhancement	17
REFERENCES	18

LIST OF FIGURES

Title	Page
Figure 1.1 Markov Decision Process	2
Figure 1.2 Increasing trend of robotic across all industries	3
Figure 4.1 Project flow diagram	8
Figure 4.2 Agent Environment Interaction	11
Figure 5.1 Initial Environment State	12
Figure 5.2 Before training action plot	13
Figure 5.2 Before training action value	13
Figure 5.3 After training action plot for agent 3 to pick object 3	14
Figure 5.4 After training action value plot of agent 3 to pick object 3	14
Figure 5.5 After training action plot to transport object 3 to goal 3	15
Figure 5.6 After training action value plot of agent 3 to transport object 3 to goal 3	15

LIST OF EQUATIONS

Title	Page
Cumulative Reward Function	9
Q-learning Update Rule	10

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
MARL	Multi Agent Reinforcement Learning
MDP	Markov Decision Process
PPO	Proximal Policy Optimization
Q-learning	Quality Learning
RL	Reinforcement Learning

CHAPTER 1: INTRODUCTION

In the past decade, machine learning has made huge strides in computer science. It can now perform tasks like recognizing images and playing games as well as, or even better than, humans. This progress has been helped by new algorithms and the sharing of benchmarks and data sets, which have pushed forward many fields.

1.1 Background

Nowadays, manual machine operation is becoming increasingly outdated in today's tech world as modern machines must adapt and learn on their own. Integrating intelligent agents will be important for improving a machine's learning ability – they will be able to identify and choose the fastest problem-solving methods. Reinforcement learning problems involve learning what to do, how to map situations to actions so as to maximize a numerical reward signal [10]. In an essential way they are closed-loop problems because the learning system's actions influence its later inputs. Simulations are essential when designing and testing Agentic applications in healthcare, aerospace, package delivery and disaster recovery as automation expands in these sectors cost-effectively yet safely. Here are some of the main concepts involved in reinforcement learning:

1.1.1 Intelligent agent

A system designed to perceive its environment through sensors and act upon it via actuators to achieve specific goals. These systems operate autonomously, making decisions and performing tasks based on their perception and inherent intelligence, which can range from simple rule-based systems to advanced machine learning algorithms. They gather information about their surroundings through interaction, process it, and take actions to meet their objectives. Intelligent agents are goal-oriented and can adapt to new situations, learning from their experiences to enhance their performance.

1.1.2 Reinforcement learning

Reinforcement learning is a type of machine learning where the agent learns from its own experiences through repeated interaction with the environment. In interactive problems it is always not possible to obtain samples of data that are both correct and representative of all the situations, like supervised learning, in which the agent has to act. In such case, the agent has to learn from its own experience. Furthermore, reinforcement learning tries to maximize a reward signal instead of trying to find hidden structure as like unsupervised learning.

1.1.3 The Agent Environment Interface and the Markov Decision Process (MDP)

The reinforcement learning scenario models the problem through the perspective of an agent learning from the environment to achieve its goal. The agent interacts with the environment and the interplay between environment and the agent's interaction continually improves the agents experience based on selecting correct actions, while maximizing cumulative sum of rewards. The environment returns a reward signal to the agent's action.

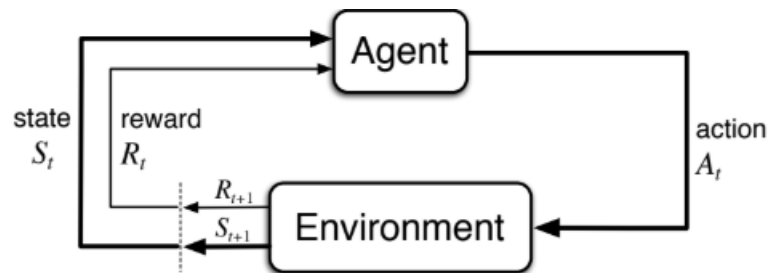


Figure 1.1 Markov Decision Process

[Source: R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction", 2nd ed. Cambridge, MA: MIT Press, 2014.]

The whole scenario of reinforcement learning is summarized by a Markov decision process defined by a tuple (Current state S_t , current action A_t , reward at time step R_t , next state S_{t+1} , future rewards R_{t+1}).

1.1.4 Reward

Reward is a numerical value that is returned from the environment as a result of agent performing action A while being in state S in the environment. Reward incentivizes the agent towards the desired action and also penalizes the agent for any unwanted actions.

1.2 Motivation

Logistic transportation is vital for daily life. It ensures the movement of goods and supplies across cities and countries, significantly impacting the economy. Advancements in reinforcement learning can enhance the efficiency and robustness of logistic transportation systems, incorporating features such as autonomous driving, advanced delivery systems and autonomous warehouse management system.

1.3 Problem Statement

Safe and effective delivery is a ubiquitous problem in the logistic transportation scenario. Conventional methods include human workers in the transportation which lacks efficiency and faster fulfilling of orders. This disrupts the smooth operation of the logistic transportation

which severely impacts on the consumption of resources and time. Due to this very cause, the intensity of use of robots has been on the rise.

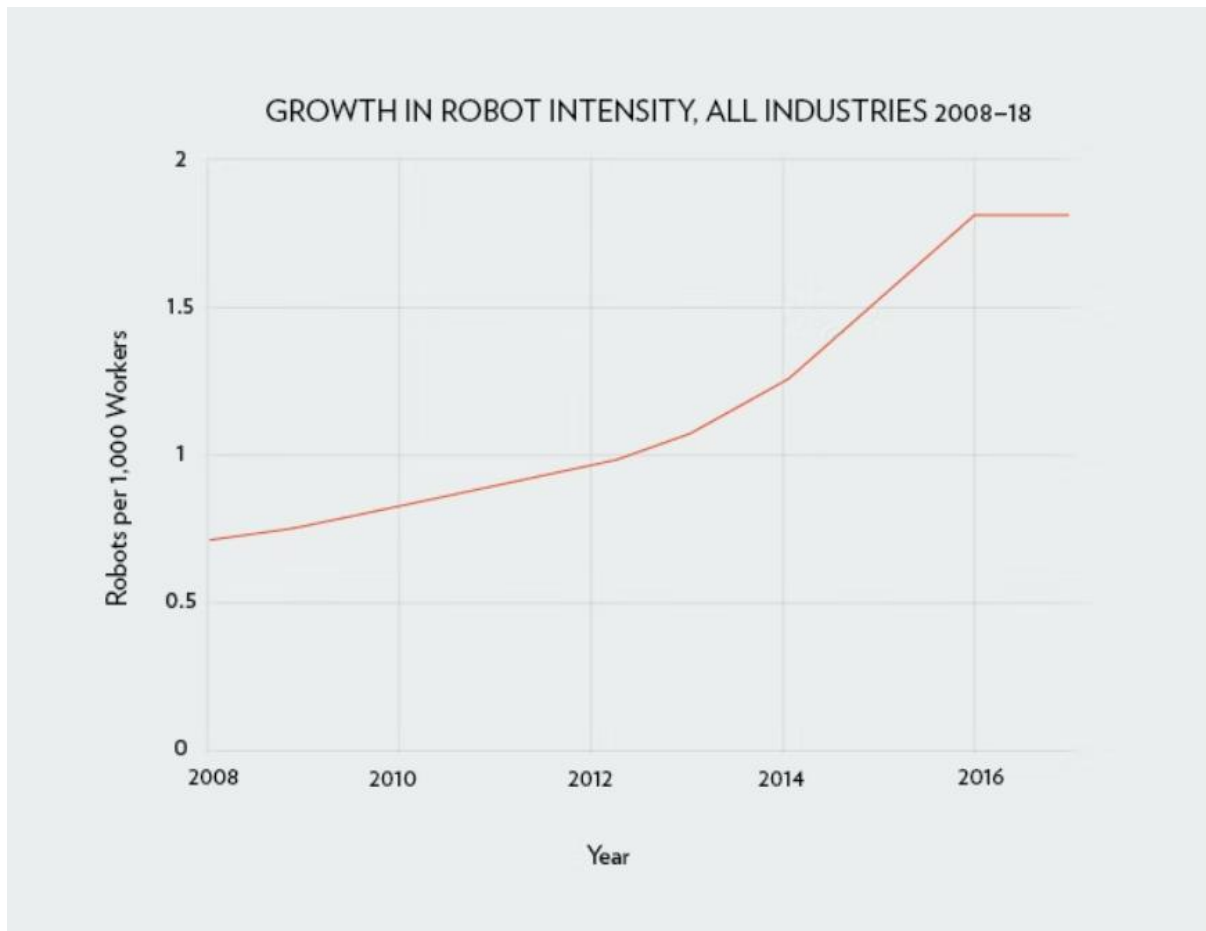


Figure 1.2 Increasing trend of robotic across all industries

[Source: William M. Rodgers III and Richard Freeman, “How Robots Are Beginning to Affect Workers and Their Wages”]

With the increasing trend of use of robots as shown in the graph, robotic automation industry has grown rapidly. However, the use and manipulation of robotics is a control problem that is sufficed by reinforcement learning.

1.4 Objective

The objectives of the project are:

- To implement effective and scalable pickup and delivery of logistics.
- To make each agent independently perform the task of pickup and delivery of objects.
- To ensure each agent should avoid the obstacles.

1.5 Significance of the study

RL can enable agents to discover novel and optimal solutions that may not be obvious or feasible for human engineers. The reinforcement learning paradigm successfully achieves the goal in the logistic warehouse problem (pickup and delivery). The issue of scalability in the multiple agent environment is solved through independent autonomous agents. In a multi agent reinforcement learning setting, it is a proof that the independent agents in the environment can successfully achieve the goal rather than using complex algorithms and frameworks.

CHAPTER 2: LITERATURE REVIEW

Littman (1994) [1] expands the Markov decision process framework to Markov games and demonstrated its efficacy in a simple two-player game where the optimal policy is probabilistic.

Comprehensive surveys for MARL techniques can be found in the works of Cherkassky and Mulier (1998) [2] and Ficici and Pollack (2000) [3]. The authors in the latter also discuss the application domains for MARL, which include multi-Agent teams. A specific relevant example that is discussed is multi-Agent object transportation.

Ming Tan (1993) [4] stated in his paper regarding independent and cooperative agents that experience sharing can help RL agents with similar tasks learn faster and reach better performance.

Hu and Yan (2022) [5] explored various Q-learning algorithms for both single-agent and multi-agent systems which included basic Q-learning, deep Q-learning, etc. and compared their strengths and weaknesses and also provided insights on their current applications and future trends.

Pitis (2019) [6] generalized traditional fixed discounting in reinforcement learning by introducing a state-action dependent discount factor. He showed that this approach broadens possible preference structures, a unique optimizing MDP with a fixed discount factor can still match the optimal policy's true utility, hence providing a justification for recent advancements in reinforcement learning formalism.

Melo and Ribeiro (2007) [7] analyzed the convergence of Q-learning with linear function approximation where they identified conditions for guaranteed convergence with probability under a fixed learning policy, comparing these results to related works and discussing the method's applicability with changing policies and in partially observable scenarios.

Busoniu, Babuška, and De Schutter (2010) [8] presented the main benefits and challenges of MARL, as well as the different viewpoints on defining the MARL learning goal. They discussed in detail a representative set of MARL algorithms for fully cooperative, fully competitive, and mixed tasks. They focused on autonomous agents that learn how to solve dynamic tasks online, with algorithms originating in temporal-difference RL, but also

investigated techniques for static tasks. Additionally, they reviewed some representative problem domains where MARL techniques have been applied and illustrated the behavior of several MARL algorithms in a simulation example involving multi-agent object transportation.

Cals et. Al [9] presented on his paper “Solving the online batching problem using deep reinforcement learning” presents a Deep Reinforcement Learning (DRL) approach together with a heuristic, for deciding how and when arrived orders are batched and picked to minimize tardy orders. Furthermore, the paper shows the formulation of the warehouse characteristics as a semi-Markov decision process and vector-based state representation allowing to create a deep reinforcement learning solution that learns strategy by interacting with the environment and solve the problem using proximal policy solution.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Functional Requirements

- The simulation environment should have a clearly defined grid or map with a starting point and a destination for the box.
- The environment should include obstacles that the agents must navigate around.
- Agents should have the ability to move in four directions (up, down, left, right).
- There should be a defined mechanism of interaction with the object like pickup.
- There should be a reward structure, such as points for moving the box closer to the destination or penalties for collisions.
- System should provide a visualization tool for observing the agent's actions and the progress of the box in the environment.

3.2 Non-functional Requirements

- The simulation should run efficiently, with minimum lag, allowing real-time observation of agent actions.
- Training time for the agents should be reasonable using optimized algorithms and hardware.
- The system should be scalable to allow more complex environments and additional agents if required.
- The system should be tolerant to minor faults and capable of recovering from errors without significant disruption to the simulation.

CHAPTER 4: METHODOLOGY

4.1 Environment setting

The environment has a grid structure, which means that every cell in the matrix can be occupied by an agent, an object or even an obstacle. Every such grid has markings on it, which are agents or objects and in turn each agent has a task to find its object's location and move it to a fixed predetermined destination avoiding the obstacles. All of these factors combined introduces an added layer of complexity to the environment. Each agent is independent of other agents, this requires decentralized training, every agent must also avoid obstacles and the overall problem requires a wide range of information to be effective - the location and the condition of every agent and object. Within this environment, agents are free to perform various movements or interactions and with every action a reward system is implemented that rewards correct decisions and punishes collisions among other things.

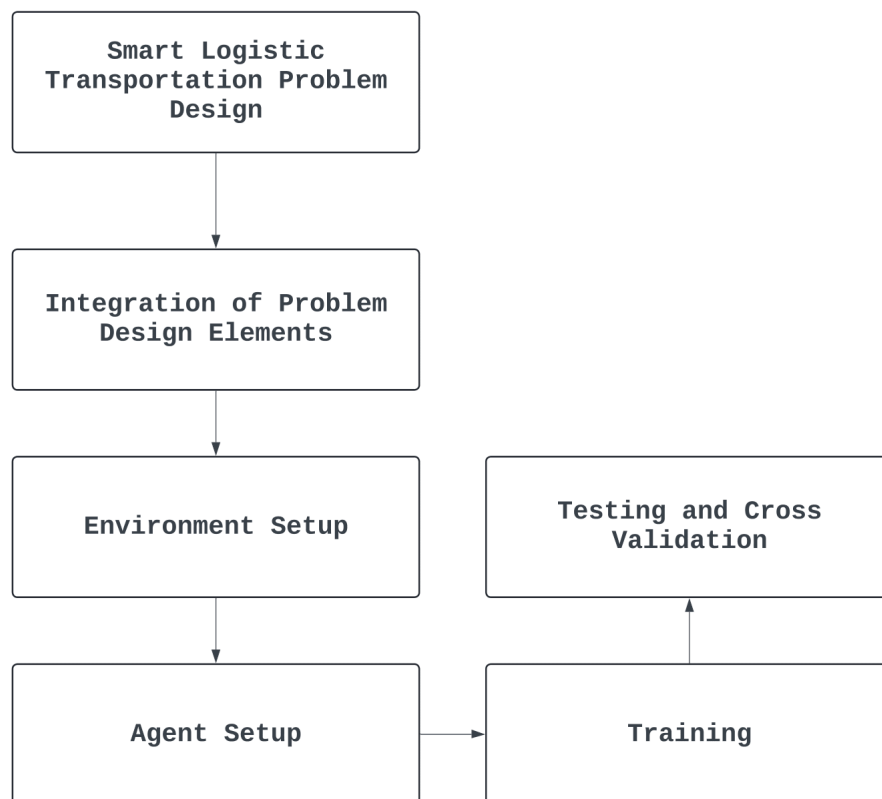


Figure 4.1 Project flow diagram

4.1.1 Single agent system

The system where a single agent interacts with the environment, and gets a reward from the environment based on its action. An agent is a specific object that decides its action based on its percept sequence and the current environment.

4.1.2 Single agent framework

In a single agent system, learning of the agent is the central theme where the agent continuously improves through trial and error. Being a sole entity, it does not interact with any other agent in the given environment. Thus, it is comparatively simpler than the multiagent system.

4.1.3 State space

State space: If the environment consists of 10 X 10 grid, then the agent can be in 100 possible locations given the absence of obstacles. The agent starts at an arbitrary position (x_{start} , y_{start}) and its task is to reach the goal position (x_{goal} , y_{goal}). It is the learning policy that drives the agent to reach the goal from any position (x , y).

4.1.4 Action space

Action space: Action space is the set of all possible legal actions that the agent can perform in a particular state. For example, if the set of available actions is “UP”, “DOWN”, “LEFT”, and “RIGHT” in the given environment, these actions constitute the action space.

4.1.5 Agent policy

The agent policy is a function which takes the current state of agent as input and returns the suitable action based on the given state.

The agent uses exploratory policy to explore set of all possible legal actions in order to achieve the goal. The agent uses exploitation policy in order to perform the task with the highest action value.

4.1.1 Reward

The agent's sole objective is to maximize the total reward it receives over the long run. The reward signal thus defines what are the good and bad events for the agent [10]. In the grid world, reward signal of -1.0 is given at each time step for not achieving the goal whereas reward signal of 0.0 is given on achieving the goal. The total reward for the agent is accumulated using the following equation:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^n R_{t+n} = \sum_{k=0}^n \gamma^k R_{t+k}$$

1

where,

G_t is the cumulative reward over time t

R_t is the immediate reward at time step t

γ is the discount factor ranging from 0.0 to 1.0

4.2 Q-learning algorithm

The project implements Q-learning algorithm. The algorithm is useful in directly approximating the optimal action-value function (q^*), independent of the policy being followed. This greatly simplifies the algorithmic analysis and enabled faster convergence. The policy still has an effect in that it determines which state–action pairs are visited and updated. The q learning algorithm is as follows:

1. Initialize $Q(s, a)$, $\forall s \in S, a \in A(s)$, arbitrarily,
and $Q(\text{terminal-state}, \cdot) = 0$
2. Repeat (for each episode):
 - 2.1 Initialize S
 - 2.2 Repeat (for each step of episode):
 - 2.2.1 Choose A from S using policy derived
from Q (e.g., ϵ - greedy)
 - 2.2.2 Take action A , observe R, S'
 - 2.2.3 $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', A') - Q(S, A)]$
 - 2.2.4 $S \leftarrow S'$
 - 2.3 Until S is terminal

Initially, all the state action value in the Q value table is initialized to 0. The main purpose of the algorithm is to converge this Q value table for an agent to learn the optimal policy. Once the algorithm is initiated, within each episode following operations are performed for the Q table update:

- The state S is rendered from the given environment.
- Then for each step of the episode, the policy function maps the rendered state to a suitable action A .
- Then following the action, the step function returns the reward R , changing to next state S' .
- The Q table is updated based on these parameters S, R, A, S' following the equation:

$$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',A') - Q(S,A)]$$

2

- Then the next state S' is assigned to current state S for the next step. This process is repeated till the end of the episode (i.e. when the goal is achieved).

For the agent to learn the optimal policy 10,000 episodes is performed.

4.3 Multi agent reinforcement learning

In multi agent reinforcement learning framework, multiple agents exist in the same environment. While various multi agent reinforcement learning algorithms exist (Proximal Policy Optimization, Multi Agent Deep Deterministic Policy Gradient, etc.), the project implements independent Q learning for the given environment.

4.3.1 Independent Q learning

Independent Q learning is an approach where each agent learns Q-functions that only depend on the current agent's action, using the basic Q-learning, and without being aware of the other agents. The environment setting for this project is fully-observable environment. Thus, independent Q learning algorithm can perform much better with multi-agent environment in fully observable setting. Another reason for the use of independent Q learning in this project is the overall simplicity.

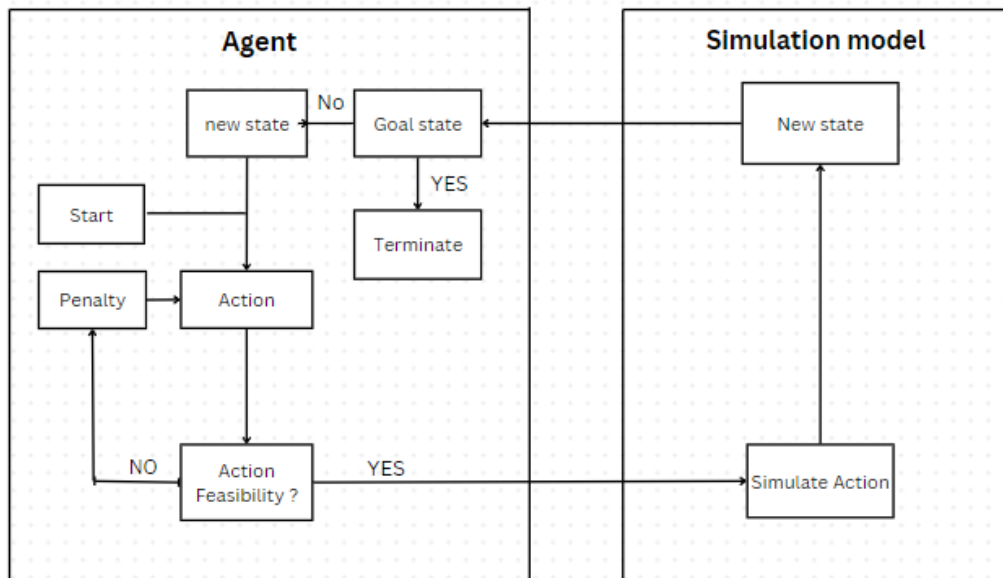


Figure 4.2 Agent Environment Interaction

CHAPTER 5: RESULTS AND ANALYSIS

5.1 Result

In this project the size of two-dimension grid maze is designed 10*10, in which three agents with their corresponding object and goals are initialized. The agents at location (0, 0), (2,2), (0, 9) and the corresponding objects at (6, 4), (7, 0), (5, 9) and goals at (9, 9), (2, 3), (9,0) respectively. Each of the agents, object and goals have unique color coordinates that differentiate each other.

To minimize an error, we run algorithm at 10000 times where each agent acts as an independent agent. This test is run using a classical Q-learning algorithm. Each agent has its own action value table which is also called Q-table which is of size (10, 10, 4) which is initially initialized with zeros and after each episode the q-value gets updated. There are 2 action value table for each agent that defines policy to drive an agent from its initial positions to object after then agent and object acts as single agent as if agent carrying object, then second action value table drive a combined agent towards goal. Similar for all agent, object and goals.

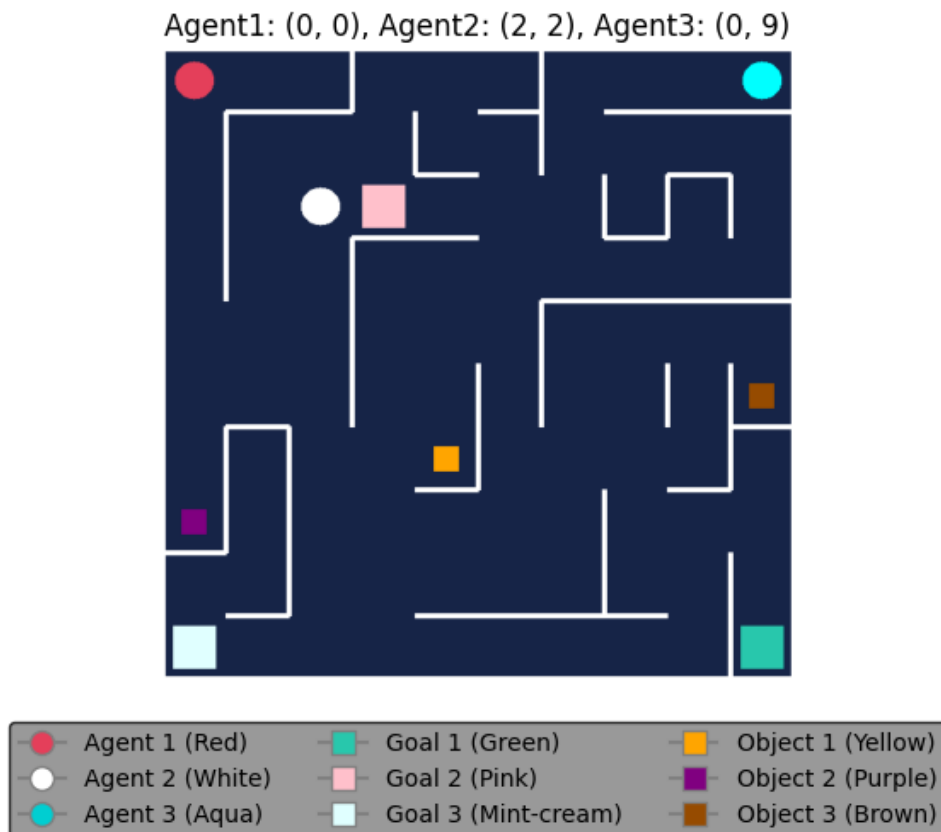


Figure 5.1 Initial Environment State

5.1.1 Before Training



Figure 5.2 Before training action plot

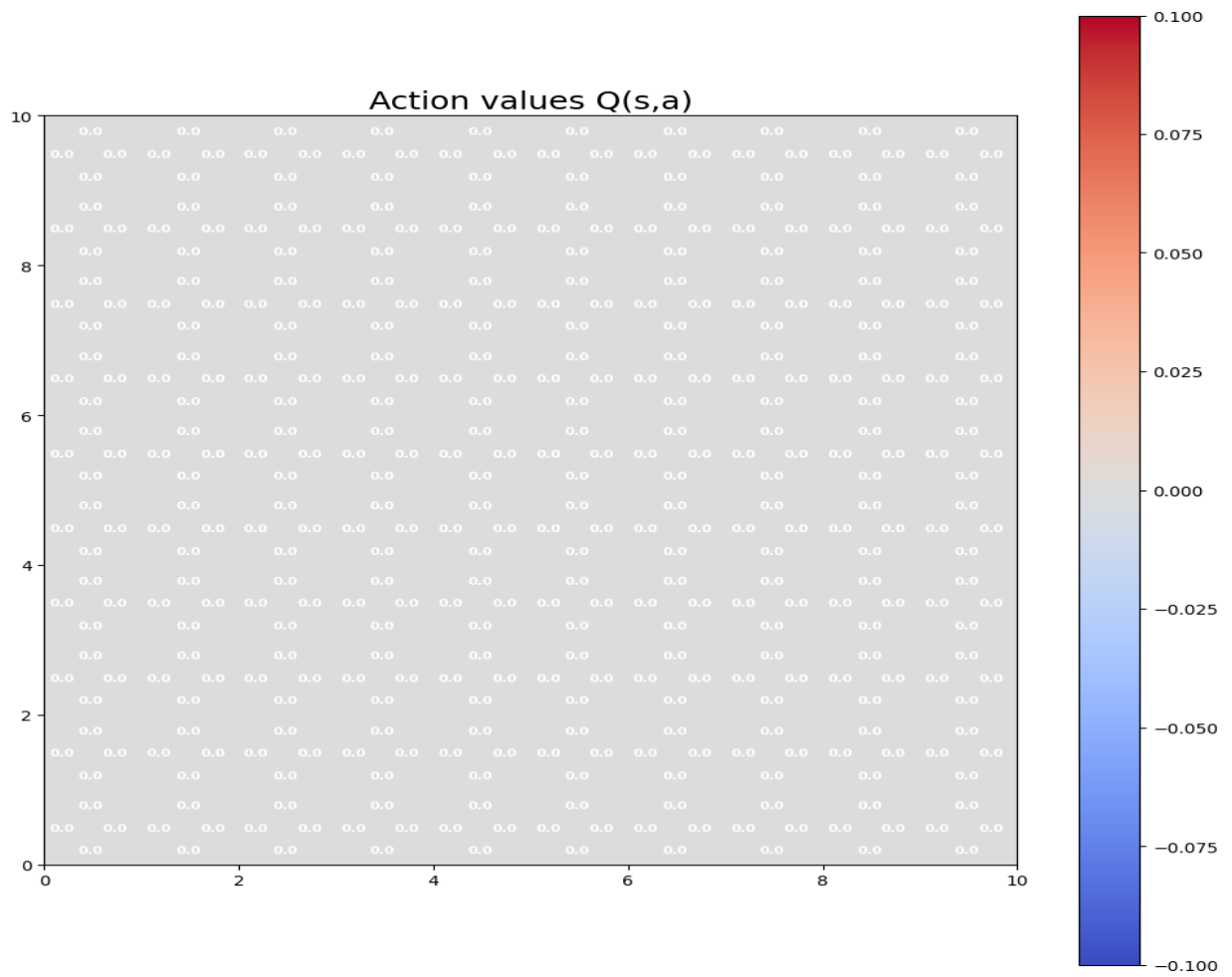


Figure 5.2 Before training action value

5.1.2 After Training

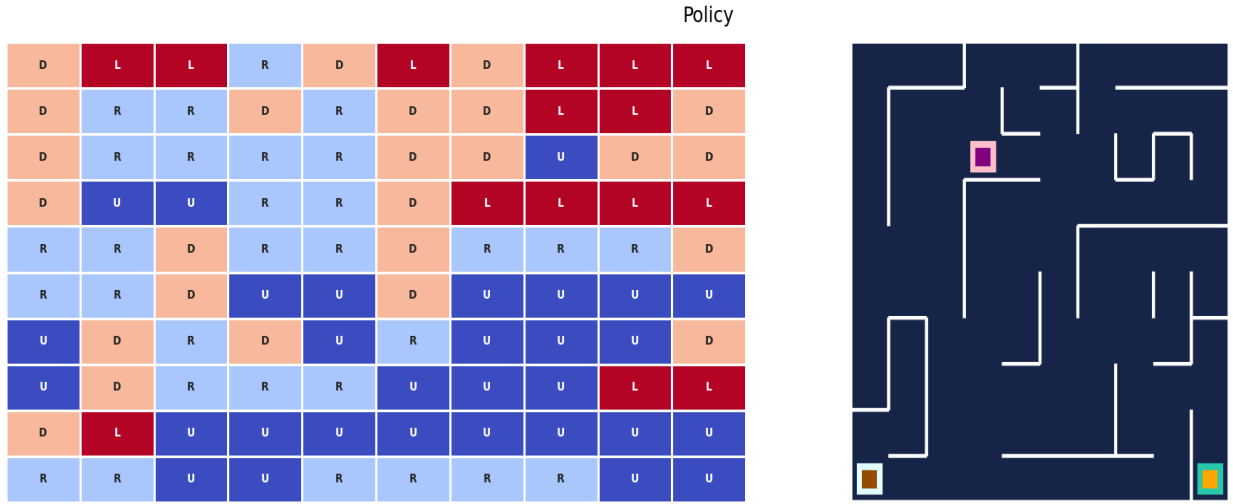


Figure 5.3 After training action plot for agent 3 to pick object 3

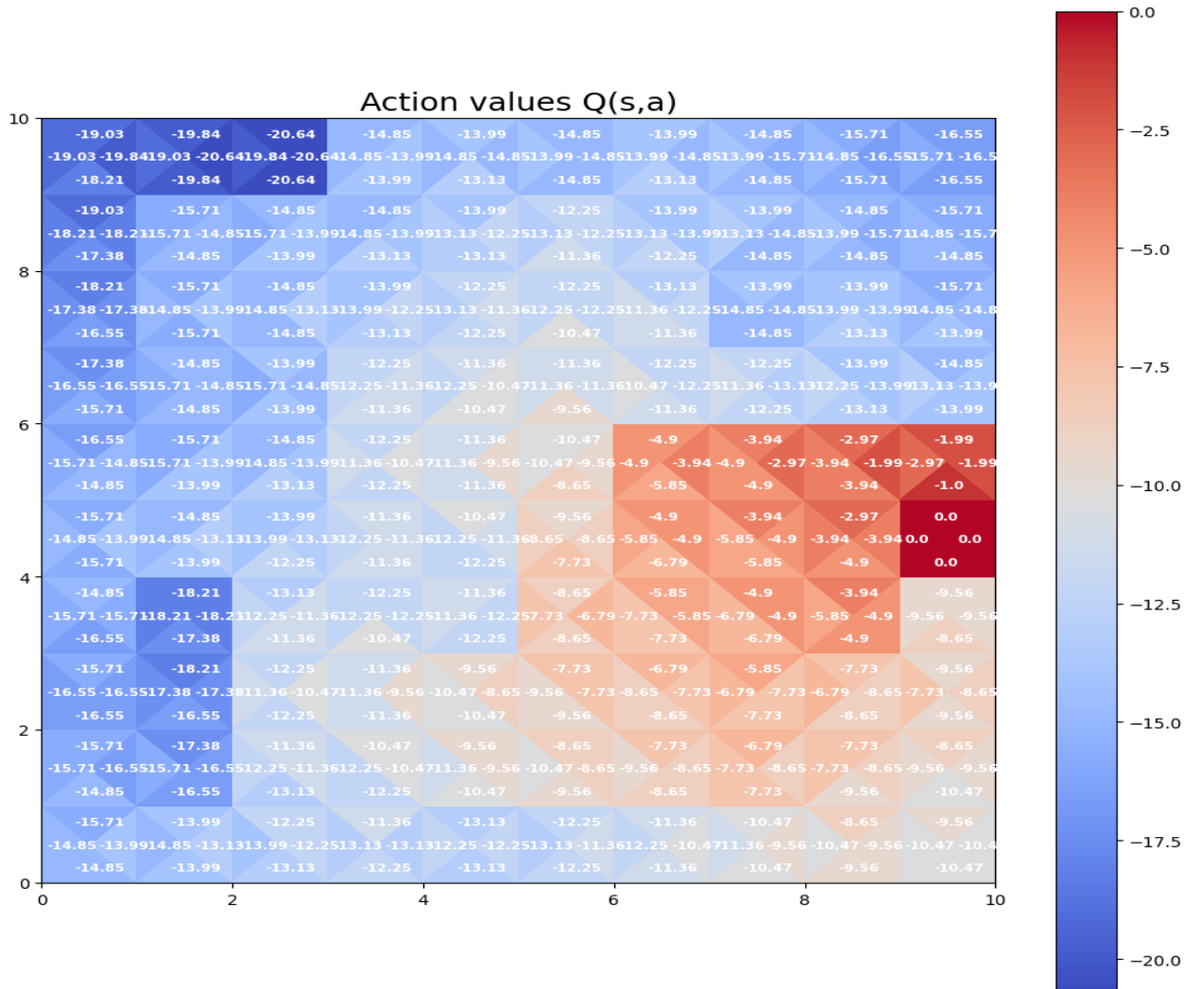


Figure 5.4 After training action value plot of agent 3 to pick object 3



Figure 5.5 After training action plot to transport object 3 to goal 3

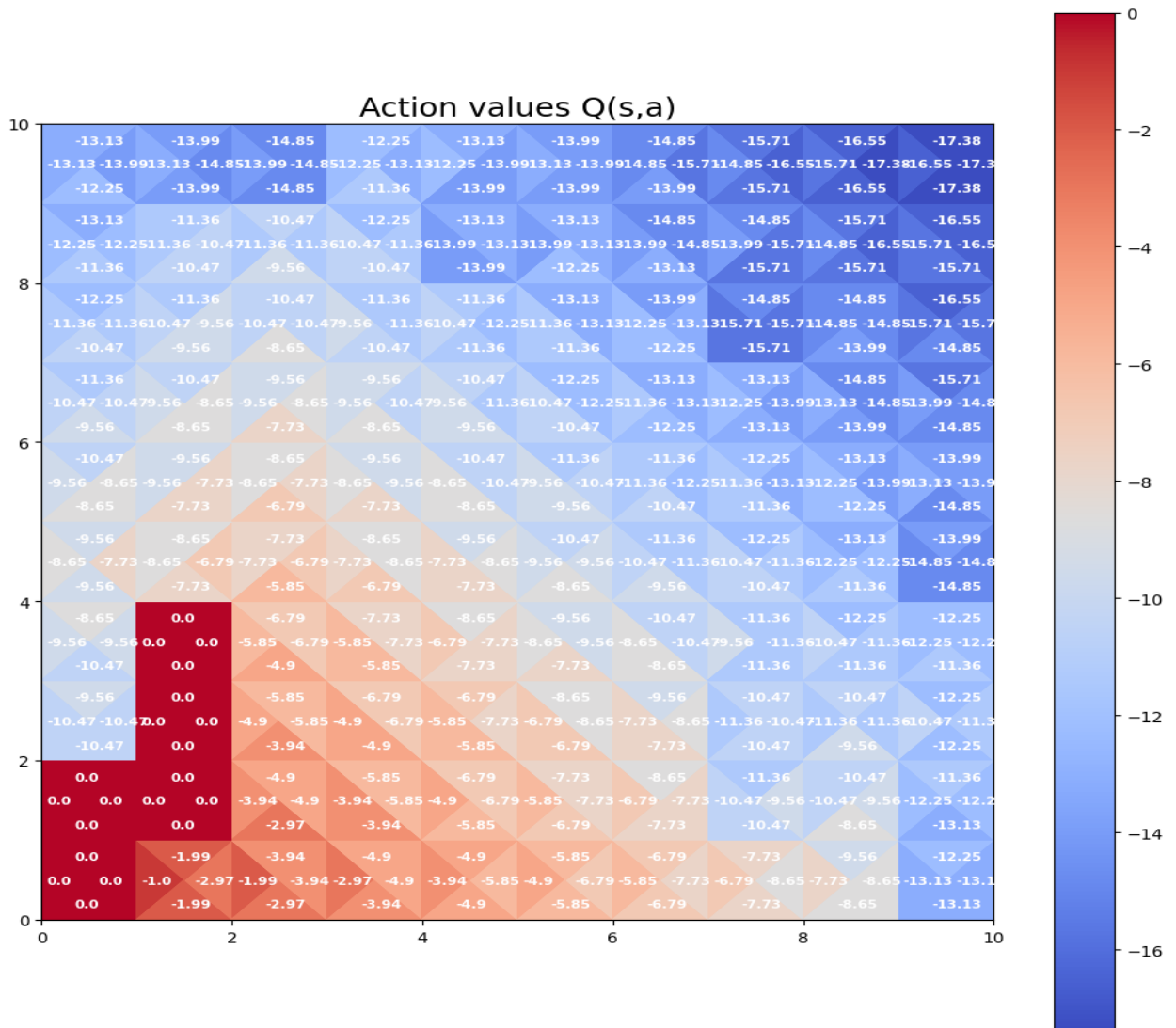


Figure 5.6 After training action value plot of agent 3 to transport object 3 to goal 3

5.2 Analysis

Initially, the agents picked random actions which led the agent to explore the overall environment. Each agent explored the environment in search of the object and goal to devise the optimal policy. Thus, the agent was seen moving or all over the spaces in the grid. Once the agent developed the optimal policy, exploitation of the optimal policy was seen i.e. the agent conducted its action based on the deduced policy which led to smooth and effective transportation of the logistic.

CHAPTER 6: CONCLUSION, LIMITATIONS AND FUTURE ENHANCEMENT

6.1 Conclusion

This report presents independent multi-agent reinforcement learning using table-based Q-learning. The task of learning optimal action policy was independent to each agent there was no agent coordination. The results had shown the agents perform on best possible policy determined after training in an environment. Agent location in a grid can be defined anywhere in case of identifying the route to its corresponding object in an environment.

6.2 Limitations

- Goal and object remain static preventing dynamic behavior.
- Increased memory usage for storing q-table for individual agent.

6.3 Future Enhancement

Some possible future enhancements of the project are listed below:

- Increasing the complexity by introducing a greater number of agents and its corresponding object and goal.
- Implementing neural network to solve generalization problem.

REFERENCES

- [1] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," Proceedings of the 11th International Conference on Machine Learning (ICML-94), 1994, pp. 157-163.
- [2] V. Cherkassky and F. Mulier, Learning from Data: Concepts, Theory, And Methods. New York: Wiley, 1998
- [3] S. G. Ficici and J. B. Pollack, "A game-theoretic approach to the simple coevolutionary algorithm," in Proc. 6th Int. Conf. Parallel Problem Solving from Nature (PPSN-VI), Paris, France, 2000, pp. 467-476.
- [4] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in Proc. 10th Int. Conf. Machine Learning (ICML-93), Amherst, US, 1993, pp. 330-337
- [5] Y. Hu and G. Yan, "Q-learning for single-agent and multi-agent and its application," in Proc. 2nd Int. Conf. Artif. Intell., Autom., High-Perform. Comput. (AIAHPC 2022), edited by L. Zhu, vol. 12348, 123482D, SPIE, 2022, doi: 10.1117/12.2642094.
- [6] S. Pitis, "Rethinking the discount factor in reinforcement learning: A decision theoretic approach," in Proceedings of the 33rd AAAI Conference on Artificial Intelligence, 2019, pp. 4603-4610.
- [7] F. S. Melo and M. I. Ribeiro, "Q-learning with linear function approximation," Technical Report RT-602-07, Institute for Systems and Agentics, March 2007.
- [8] L. Busoniu, R. Babuška, and B. De Schutter, "Multi-agent reinforcement learning: An overview," in Innovations in Multi-Agent Systems and Applications - 1, D. Srinivasan and L. C. Jain, Eds. Berlin, Germany: Springer, 2010, ch. 7, vol. 310, Studies in Computational Intelligence, pp. 183-221.
- [9] B. Cals, Y. Zhang, R. Dijkman, and C. van Dorst, "Solving the online batching problem using deep reinforcement learning," Computers & Industrial Engineering, vol. 156, p. 107221, Mar. 2021.
- [10] R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction", 2nd ed. Cambridge, MA: MIT Press, 2014.